

Debugging, Profiling, Tracing et Analyse de performance sous Linux

Tracing, réseau et méthodologie

eBPF, bpftrace, ss/tcpdump et démarche d'investigation

Objectifs du module

À la fin de ce module, vous serez capable de :

- Poser le vocabulaire eBPF et situer bpftrace
- Analyser une performance réseau avec ss, tcpdump, iftop
- Dérouler une méthodologie d'incident structurée
- Choisir le bon outil au bon niveau d'abstraction

Plan

1. Tracing et introduction à eBPF / bpftrace
2. Analyse de performance réseau
3. Méthodologie et bonnes pratiques
4. Démo et TP final
5. Récap et questions

1. Tracing et eBPF

Pourquoi le tracing

“ strace voit les syscalls d'un process, perf voit les pics CPU. Le tracing répond à « que se passe-t-il dans le noyau, pour qui et combien de fois ? ». ”

Usages typiques :

- Mesurer la latence d'un appel par percentile
- Compter les écritures disque par process sur 10 minutes
- Suivre une requête traversant plusieurs couches

eBPF en 2 minutes

- Mini-machine virtuelle dans le noyau
 - Programmes attachés à des sondes (kprobe, uprobe, tracepoint, USDT)
 - Vérifiés avant exécution pour garantir la sûreté
 - Overhead très faible, utilisable en prod
- bpfttrace est un langage de haut niveau qui compile en eBPF.

bpfttrace : exemples essentiels

```
# Histogramme de latence des lectures VFS
bpfttrace -e 'kprobe:vfs_read { @start[tid] = nsecs; }
              kretprobe:vfs_read /@start[tid]/
              { @us = hist((nsecs - @start[tid]) / 1000);
                delete(@start[tid]); }'

# Top des commandes qui exec
bpfttrace -e 'tracepoint:syscalls:sys_enter_execve
              { @[comm] = count(); }'
```

2. Analyse réseau

ss pour l'état des sockets

```
# Sockets TCP établies, avec stats
ss -tni

# Sockets en écoute, avec process
sudo ss -tlnp

# Sockets en CLOSE_WAIT (bug applicatif fréquent)
ss -tn state close-wait
```

ss remplace netstat, plus rapide et plus riche.

tcpdump pour la capture

```
# Capture filtrée sur port, limite 100 paquets
sudo tcpdump -i any -c 100 port 443

# Capture vers fichier pour wireshark
sudo tcpdump -i eth0 -w capture.pcap 'host 10.0.0.5'

# Lire un pcap
tcpdump -r capture.pcap -nn
```

- Toujours filtrer : sans filtre, volume ingérable
- `-nn` évite les résolutions DNS qui masquent le vrai problème

iftop et vue débit

```
sudo iftop -i eth0
```

- Vue temps réel par couple src/dst
- Utile pour détecter un flux dominant
- Alternative légère : `nload` ou `bmon`

3. Méthodologie

USE method (Utilization, Saturation, Errors)

Pour chaque ressource (CPU, mémoire, I/O, réseau), poser trois questions :

1. Quelle est son utilisation ?
2. Est-elle saturée (file d'attente) ?
3. Y a-t-il des erreurs ?

Cette grille évite de sauter trop vite à une conclusion.

Démarche d'incident

1. Stabiliser : qu'est-ce qui est cassé et pour qui
2. Observer : top, vmstat, iostat, dmesg, logs
3. Corréler : corrélation temporelle avec déploiements ou pics
4. Hypothèse : formuler une cause candidate testable
5. Instrumenter : strace, perf, bpftrace sur la cible
6. Conclure : preuve, pas intuition
7. Documenter : post-mortem réutilisable

Limites et risques en prod

- strace ralentit fortement, éviter sur un process critique saturé
- tcpdump sans filtre peut saturer le disque
- perf record longue durée = gros volumes
- bpftrace sur tracepoint hot path peut avoir un coût non nul
- Toujours avoir une porte de sortie (timeout, `sleep N`, fenêtre courte)

Exercice et TP

Exercice 5 : choisir le bon outil pour une série de symptômes

- Fichier : `exercices/exercice-05.md`

TP 5 : étude de cas de dépannage de bout en bout

- Durée : 1h30
- Fichier : `tp/tp-05.md`

